Capítulo 3

Vetores Não-Ordenados – Busca Sequencial

Exercícios

1. (ENADE): Considere um algoritmo de busca sequencial que procura por um determinado valor em um vetor de tamanho n. Sobre a complexidade desse algoritmo, assinale a alternativa **CORRETA**:

A. Melhor caso em tempo O(n) e pior caso em tempo $O(n^2)$.
B. Melhor caso em tempo O(1) e pior caso em tempo O(n).
C. Melhor caso e pior caso ambos O(log n).
D. A complexidade independe de n (tempo constante, O(1)).

2. (TEÓRICA): Suponha que você tenha uma lista não ordenada com 100 nomes e deseja verificar se o nome "Mariana" está presente. Sobre o algoritmo de busca sequencial nesse contexto, é **correto afirmar** que:

A. No pior caso, será necessário comparar "Mariana" com **todos** os 100 nomes da lista.
B. Caso "Mariana" não esteja na lista, o algoritmo irá interromper a busca na metade, por eficiência.
C. Se "Mariana" estiver na primeira posição, ainda assim serão verificadas todas as 100 posições para confirmar duplicatas.
D. A busca sequencial não funciona com listas de strings, apenas com números, devido às comparações.

3. (TEÓRICA): Considere a implementação clássica da busca sequencial que retorna o índice da primeira ocorrência do valor alvo ou -1 se não encontrar. Dada uma lista `L = [7, 12, 5, 12, 8]`, se chamarmos a função `busca_sequencial(12, L)`, qual será o resultado retornado?

A. 1
B. 2
C. 3
D. -1

4. (PRÁTICA): Implemente uma função em Python chamada `buscar_cliente(nome, lista_clientes)` que realiza busca sequencial em uma lista de nomes de clientes. Essa função deve retornar o índice onde o `nome` foi encontrado ou `-1` caso o nome não esteja na `lista_clientes`. Em seguida, escreva um pequeno código de exemplo que utilize essa função para buscar por pelo menos um nome que exista e um que não exista na lista, exibindo mensagens adequadas.

5. (PRÁTICA): Crie uma função `conta_ocorrencias(valor, lista)` em Python que percorre uma lista e conta quantas vezes um determinado valor aparece. Utilize busca sequencial (varrendo a lista) para realizar a contagem. Por exemplo, `conta_ocorrencias(3, [1,3,5,3,7])` deve retornar 2, pois o valor 3 aparece duas vezes na lista. Inclua casos de teste chamando a função com diferentes valores e listas.

6. (PRÁTICA): Suponha que precisamos remover um elemento específico de uma lista não ordenada. Escreva uma função Python `remove_elemento(lista, valor)` que remove **a primeira ocorrência** de valor na lista, mantendo os demais elementos e retornando o valor removido. A função deve retornar `None` caso o valor não seja encontrado ou a lista esteja vazia. ***Dica:*** *Utilize a busca sequencial para encontrar a posição do elemento e, se encontrada, deslize os elementos posteriores para "preencher" o espaço removido (ou use métodos próprios da lista Python). Demonstre o funcionamento da função com um exemplo.*

Projetos Propostos

Projeto 1: Encontrando seu Filme Favorito

Contexto: Imagine um sistema de recomendação de filmes simples onde os filmes são armazenados em uma lista não-ordenada.

Problema: Implementar uma função que permita ao usuário buscar um filme pelo título nessa lista.

Indicação de Solução: *Crie uma lista de títulos de filmes e uma função que use a busca sequencial para encontrar um título inserido pelo usuário. Se o filme for encontrado, exiba uma mensagem informando o título e sua posição (índice) na lista. Se não for encontrado, exiba uma mensagem informando que o filme não está no catálogo.*

Projeto 2: Verificando a Lista de Convidados

Contexto: Desenvolver um programa para gerenciar a lista de convidados de um evento.

Problema: Implementar uma funcionalidade que permita verificar se um nome específico está na lista de convidados.

Indicação de Solução: *Crie uma lista de nomes de convidados. Implemente uma função que receba um nome como entrada e use a busca sequencial para verificar se esse nome existe na lista. Exiba uma mensagem indicando se o convidado está na lista ou não.*